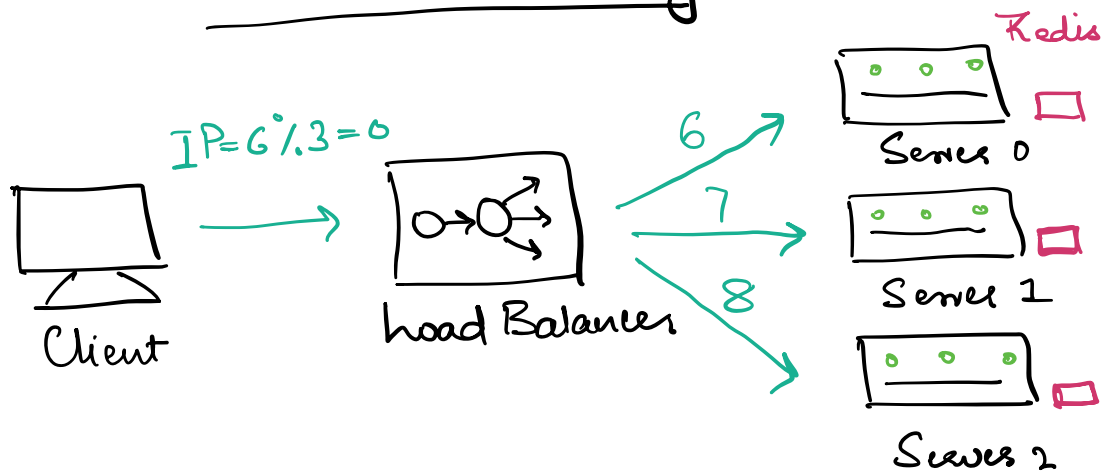


Consistent Hashing



In case of Hashing say if the user has $IP = 6$ so what hashing does is it takes the uniform IP and divides it with the no of servers the remainder that we get is the server no where the req should go like in case of $IP = 6$ the remainder of $6 \% 3 = 0$ so the req is mapped to server 0.

Meaning the req from $IP = 6$ will always go to 0 or $IP = 7$ will always go to server 1 this is called Consistent Hashing

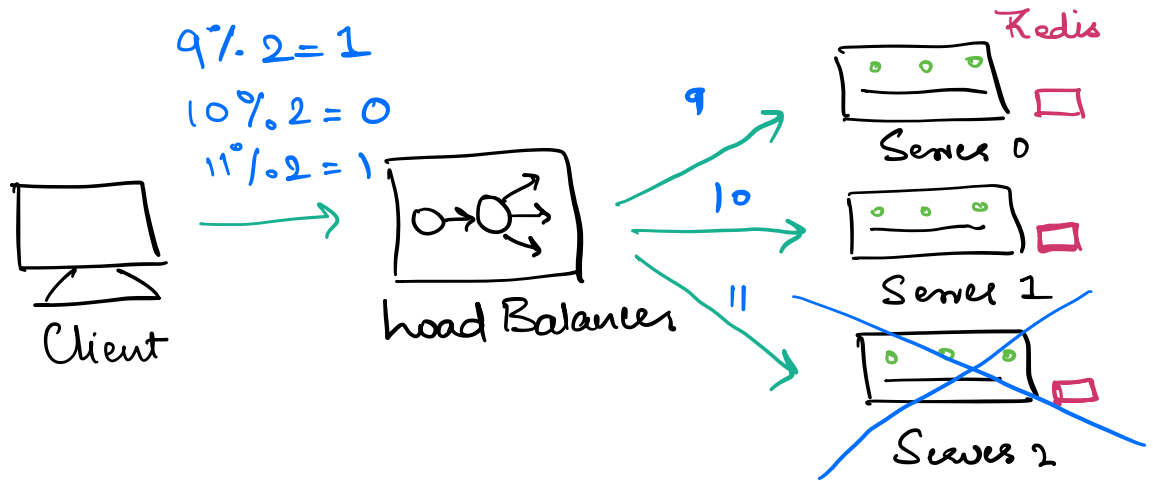
Benefit of Consistent Hashing :-

Want individual req to be mapped to the same server

In case if we want to cache the data say in Redis (In memory cache of the server) so in this case the req that come to a given server will only be cached
Eg:- Server 0 will only cache req from $IP = 6$ and not of $IP = 7$. So the next time $IP = 6$ req the same Server 0 it can get the details from cache unlike the Round Robin

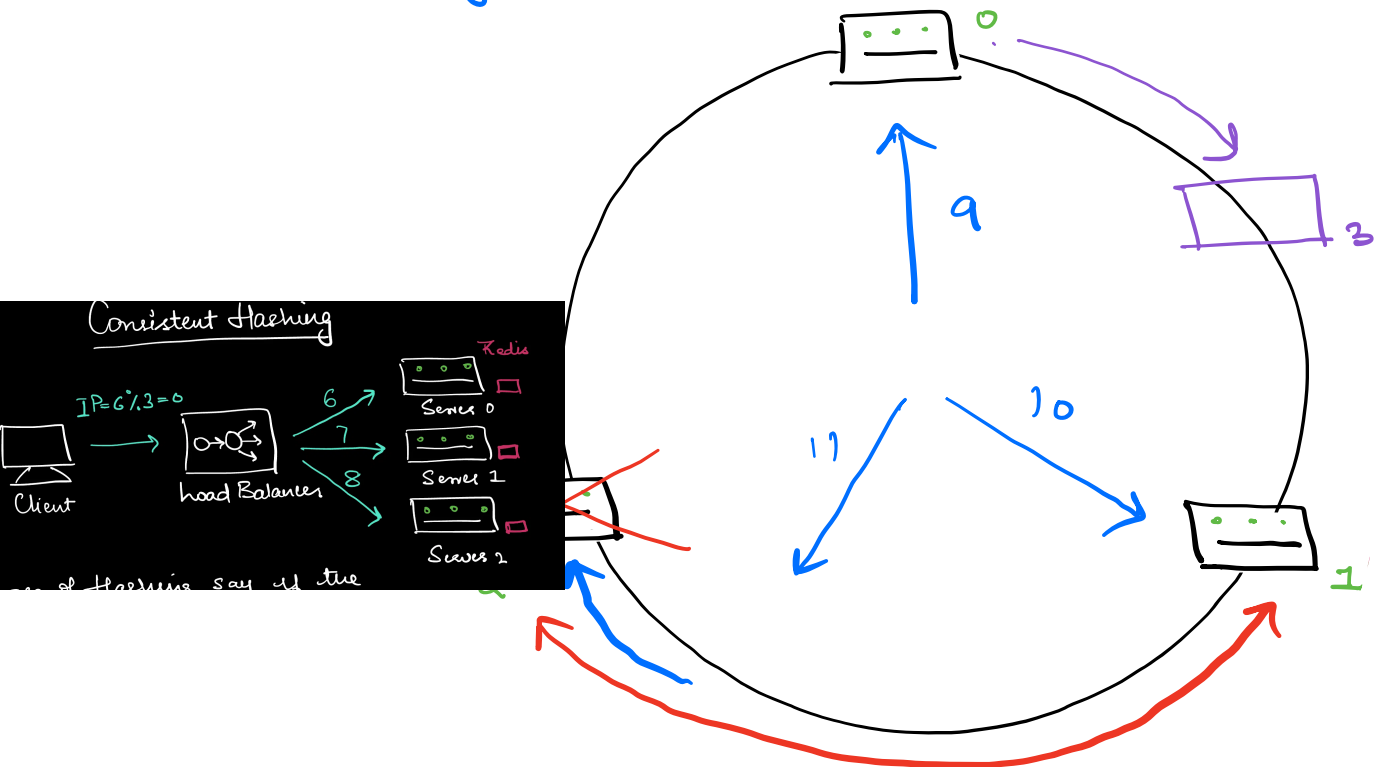
✗ CH used in case where Cache is not shared with other server

Issue with Consistent Hashing



Now, in case if Server 2 crashed it won't be like now the req to Server 2 only will get re-routed to Server 0 or 1 only the request mapping would change coz $9 \% 2 = 1$ & $10 \% 2 = 0 \dots$

So the data cached for 9 in server 0 is useless becoz of server 2 crash the req now will go to Server 1



Mapping the user req on this circular space using Hashing $IP \% 360$

The users after hashing getting mapped into the red region were going to Server 2 but now since the Server 2 crashed now these users will continue to go in clockwise and get routed to Server 0

In this whatever request are getting routed to Server 0 & 1 won't change only the Server 2 req will go to Server 0. Still it does overcrowd Server 0 with more no of req almost $2/3^{\text{rd}}$ but there are ways to handle that overload. Also

Same way, if we add a new Server 3 as shown the area highlighted in purple will go to Server 3 or any additional req may use Server 3 if we do not want to disturb the original reqs that we routed to Server 1. Rest all reqs around the circle will remain unchanged.

Consistent Hashing High level

Pros

- 1) Used if we want the users to be consistent hashed/mapped to the same server incase if the cache is no shared.
- 2) CDNs - Useful to route req to same CDN

Cons

- 1) It can get complex

3) Databases - If distributed
say user data is on
1 db so that req should go
to same DB routing to other
DB won't help

⇒ Rendezvous Hashing is similar to Consistent Hashing
algorithm no much difference

* Components of Consistent Hashing

- Hash Key (IP address)
- Hash Function (SHA, MD5, bcrypt)
- Nodes (Servers, CDNs, DB partition/shards)